Plataforma de conciencia cibersituacional

Javier Saguar Antona · ETSIT-UPM · TFG 2026–27

Pipeline de datos en tiempo real que convierte telemetría cruda en **incidentes correlacionados, contextualizados y priorizados con explicación**. Tres pilares: **A** detección multi-etapa (CEP) · **B** grafo de ataque razonado (Datalog) · **C** priorización explicable.

El problema y la idea

- **Fatiga de alertas**: el SOC recibe más alertas de las que puede revisar. Dato propio: **una regla genera 292 alertas (39 %) en APT29** por *una sola* acción del atacante.
- **Sin contexto**: la alerta no dice qué técnica es, quién la usa ni qué activos peligran.
- **Ataques multi-etapa**: la evidencia es la *secuencia*, no el evento suelto.
- **Aportación = integración**: unir detección de secuencias en tiempo real + razonamiento sobre rutas con evidencia observada + priorización justificada. Encaja con la conciencia situacional de Endsley: *percibir* (A) · *comprender* (B) · *proyectar* (C).
- **Núcleo defendible**: A + B + evaluación. Validación con datasets públicos con **verdad conocida** (MITRE Evals APT29, OTRF) y un laboratorio propio.

Los tres pilares

Pilar	Qué aporta	Fase
A · Multi-etapa	Detecta *secuencias* de la kill chain con CEP, no alertas sueltas	F3
B · Grafo	Rutas factibles hacia los activos críticos, con trazabilidad lógica	F5
C · Priorización	Puntuación descompuesta en factores + un «por qué» legible	F6

Qué NO es: no es un EDR (consume telemetría), no compite en cobertura con un SIEM comercial y hoy **no ejecuta respuesta**: es soporte a la decisión (la propuesta del agente autónomo revisaría esto).

Recorrido del dato

Datasets → **Replay** → `raw.host.windows` → **Normalización OCSF** → `norm.events` → **Detección Sigma** → `det.alerts` → **CEP Flink** → `det.campaigns` → *grafo (B)* → *priorización (C)* → *API* → *portal*. Todo se persiste en **ClickHouse**; lo no procesable va a `dlq.*` con su motivo (nada se descarta en silencio).

Las fases

F0 · Entorno — hecha

WSL2 + Docker Engine, dos repos (código y memoria), hook que garantiza la autoría de los commits.

F1 · Ingesta, OCSF, persistencia, observabilidad — hecha

Replay de datasets a Redpanda; normalización a OCSF de ∼35 tipos de evento de Windows (**100 %** en el dataset de referencia, **99,4 %** en APT29); señales precalculadas (`signal_*`) como semilla de la explicabilidad; ClickHouse; Prometheus + Grafana declarativos.

F2 · Detección con reglas Sigma + ML-1 — hecha (30-07)

Pipeline Sigma→OCSF **propia** (la oficial tenía 3 desviaciones); evaluador en memoria **73**× más rápido. APT29: **741 alertas**, 4 hosts, 33 técnicas. Línea base ingenua 741→87 (88,3 %). Cobertura ATT&CK: 300 técnicas posibles, 34 observadas. ML-1 (Isolation Forest) AUC 0,995, pero el **control de solo volumen da 0,941**: separabilidad en parte artefacto del dataset. Laboratorio propio: 14.895 eventos etiquetados.

F3 · Pilar A — detección multi-etapa (CEP) — en curso

Corpus reproducible (530.401 eventos, 9 datasets, 1.184 alertas). Cada patrón: **oráculo en batch** (ClickHouse) y luego **streaming** (Flink). **P1 robo de credenciales**: batch y Flink, **7 de 7** variantes LSASS (ancla `T1003.*`; con `T1003.001` solo 6, falla Mimikatz). Hallazgo: `MATCH_RECOGNIZE` no expresa rachas abiertas → **correlación en dos niveles**. **P2 movimiento lateral**: en batch, salto SCRANTON→NASHUA (17 aristas Sysmon 3, confianza 1,0). Arreglados: identidad de evento, hora de recogida (hasta 101 s de error), particionado. *Falta*: P2 en Flink, P3, P4, falsos positivos, latencia.

F4 · Plano CTI + extracción de TTPs (ML-2) — pendiente

OpenCTI (STIX 2.1) con ATT&CK y feeds de IOCs. ML-2: afinar SecureBERT para pasar de informes en lenguaje natural a técnicas ATT&CK (encaja con la línea NLP del grupo). Plan B: ATT&CK directo a Neo4j.

F5 · Pilar B — grafo de ataque razonado (ML-3) — pendiente

Neo4j con hosts, cuentas, servicios, vulnerabilidades, alertas y CTI; analítica GDS (centralidad, caminos). Reglas **Datalog estilo MulVAL** en Soufflé que usan la **evidencia observada**. ML-3: predicción de enlaces con GNN; *el modelo propone, Datalog verifica.*

F6 · Pilar C — priorización explicable + API (ML-4) — pendiente

Puntuación = suma ponderada de factores legibles (confianza, táctica, criticidad del activo, cercanía en la ruta, CTI, EPSS, anomalía) + narrativa. ML-4: aprender los pesos (con SHAP). FastAPI. **Propuesta nueva**: agente de respuesta autónoma acotada (F6.4).

F7 · Evaluación cuantitativa — pendiente

Protocolo escrito *antes*. Detección, reducción de alertas, priorización, razonamiento, rendimiento y ML, contra 4–5 líneas base (solo Sigma, sin anomalías, severidad de regla, Wazuh. . .).

F8 · Portal web (extra) — base hecha

React con datos **simulados**: situación, grafo del pipeline, cuadros de Grafana y demo de conciencia situacional con asistente de respuesta (*las reglas deciden, el LLM solo redacta*).

F9 · Despliegue, memoria y defensa — pendiente

Kubernetes local (minikube/kind), `make demo`, memoria y presentación con vídeo.

Hitos: **30-nov** F3 funcionando · **28-feb** núcleo (F3 + F5 simbólico), punto de no retorno · **15-may** memoria a la tutora · **jun-2027** defensa. Si el calendario aprieta, el ML se recorta en orden ML-4 → ML-1 → ML-3 → ML-2.

Tecnologías: qué son y por qué se usan

Por qué cada pieza, en una línea

Tecnología	Qué es	Por qué esta	En vez de
Datos y estándares de seguridad			
MITRE ATT&CK	Catálogo de tácticas, técnicas y subtécnicas de atacantes reales	Idioma común entre reglas, datos, CTI e informes; el orden de tácticas da la progresión del ataque	—
OCSF	Esquema abierto para normalizar eventos de seguridad (`class_uid`, `unmapped`)	Desacopla la detección del formato de origen: una fuente nueva = un mapeador, sin tocar reglas	formato nativo
Sigma + pySigma	Formato abierto de reglas + biblioteca que las adapta a un modelo de datos	Miles de reglas de la comunidad ya etiquetadas con ATT&CK; pipeline OCSF propia (la oficial tenía 3 fallos)	reglas propias
Sysmon	Telemetría ampliada de Windows (procesos, red, ficheros)	Es la fuente de los datasets; el EID 3 da la arista de red para el movimiento lateral	solo logs nativos
OTRF / MITRE Evals	Datasets públicos de ataques grabados (APT29, LSASS)	**Verdad conocida**: se sabe qué hizo el atacante, cuándo y dónde	datos reales
Atomic Red Team + VirtualBox	Pruebas atómicas de técnicas en una VM aislada	Actividad benigna sostenida y técnicas dirigidas para medir falsos positivos	malware real
Ingesta, procesado y almacenamiento			
Redpanda	Bus de eventos compatible con la API de Kafka	Binario único sin JVM ni ZooKeeper: ligero en un portátil; topics `raw→norm→det→enr`	Kafka
Python + confluent-kafka	Replay, normalización, detección y sinks	Rápido de desarrollar; la lógica de mapeo es la misma en batch y en streaming	PyFlink en esa etapa
Apache Flink 2.2.1	Motor de streaming con estado y tiempo de evento	El CEP necesita estado, tiempo de evento y marcas de agua; Flink es streaming nativo	Spark (micro-lotes)
SQL `MATCH_RECOGNIZE`	Reconocimiento de patrones de filas (SQL:2016)	Declarativo y usable sin Java; validado con prueba de humo antes de comprometerlo	Pattern API (Java)
ClickHouse	Base de datos columnar OLAP	Agregaciones rápidas sobre millones de eventos; aloja los oráculos en batch; `ReplacingMergeTree` deduplica por identidad	Cassandra, Elastic
Razonamiento e inteligencia (fases futuras)			
Neo4j + GDS	Base de datos de grafos de propiedades + analítica	Estándar de facto en seguridad (BloodHound); centralidad, caminos y comunidades	RDF / SPARQL
Datalog (Soufflé), estilo MulVAL	Lógica: hechos + reglas → estados alcanzables	Enfoque canónico en grafos de ataque; **cada conclusión trae su árbol de derivación** (explicable)	motor de reglas, OWL
OpenCTI / STIX 2.1	Plataforma y lenguaje estándar de inteligencia de amenazas	Grafo CTI nativo con conector de ATT&CK y feeds de IOCs	MISP solo
SecureBERT / TRAM	Modelos de lenguaje del dominio de seguridad	Extraer técnicas de informes; cabe en la GPU del portátil (∼110M parámetros)	LLM grande
Aprendizaje automático			
scikit-learn (Isolation Forest, RF, SMOTE)	Detección de anomalías por ventana de host	Detecta lo inusual sin regla; **no genera alertas**, eleva la confianza de una campaña	—
MLflow	Registro de experimentos	Reproducibilidad: parámetros, métricas y semillas de cada ejecución	hojas sueltas
Plataforma, observabilidad e infraestructura			
Prometheus + Grafana	Métricas y cuadros de mando	Observabilidad de cada pieza; cuadros aprovisionados desde ficheros (reproducibles)	—
FastAPI / Dagster	API REST / orquestación por *assets*	Asíncrono, tipado y con OpenAPI / moderno y orientado a datos	Flask / Airflow
React + Vite + Tailwind	Portal web del analista (extra)	Stack ya probado en otra asignatura; boceto para enseñar hacia dónde va	React 18 + Cytoscape
Docker en WSL2 (perfiles)	Todos los servicios en contenedores en un solo host	Sin sobrecoste de una VM por servicio; se levanta solo lo que pide cada fase	VMs, nube

Principios de diseño

- **El modelo propone, la lógica dispone**: ningún componente opaco decide solo (ML, LLM).
- **Oráculo antes que streaming**: cada patrón se valida contra su versión en batch.
- **Medir antes de decidir**: ventanas, anclas y particionado salen de los datos.
- **Nada se descarta en silencio**: DLQ con motivo; cada cifra con su procedencia.

Cifras para tener a mano

- 530.401 eventos · 1.184 alertas · 741 en APT29 · 12 campañas (11 P1 + 1 P2).
- P1: **7/7** variantes LSASS · P2: SCRANTON→NASHUA, 17 aristas, 516 s.
- Reducción 0,991 **solo en el alcance de P1**: *no* comparable con el 88,3 %.
- Falsos positivos de los patrones y latencia: **aún sin medir**.

Propuesta para hablar: agente de respuesta autónoma acotada (F6.4)

Hoy el asistente **recomienda** (aislar equipo, bloquear cuenta, cortar el C2) y decide el analista. Propuesta: que un agente **ejecute** acciones reversibles y de bajo coste por sí mismo, dentro de una **política declarativa** (Datalog) y con catálogo cerrado de acciones; el LLM nunca decide (la telemetría la escribe el atacante). Argumento: en APT29 pasan **4 min 7 s** entre el acceso a LSASS (P1, confianza 0,85) y el salto a NASHUA. Requiere revisar la decisión «solo soporte a la decisión» y evaluarlo en el simulador frente a un analista con distintas latencias.